Devenir opérationnel en Go

claranet®

Formation Go 2026

Objectifs du cours

Comprendre et connaître les fondamentaux

Être capable de réaliser des applications

Être capable d’approfondir en autonomie

Qui je suis

Historique de Go

Historique sur les origines de Go

Installation de Go sur sa machine

Création d’un premier binaire

Limite des CPU

Modèle	Cœurs	Threads	Fréquence	Cache L1	Cache L2	Cache L3	Date
Xeon E3-1290	4	8	3,6 GHz	4×64 Kb	4×256 Kb	8 MB	2012
Xeon E5-2697	18	36	2.3 - 3.6 GHz	28x64 Kb	14x256 Kb	35 MB	2015
XEON® P 9282	56	112	2.6 - 3.8 GHz	56x64 Ko	56x1 Mo	77 MB	2019
XEON® P 8490H	60	120	1,9 - 3,5 GHz	60x48 Kb	60x2 Mo	112,5 MB	2023
XEON® 6 6978P	120	240	2,1 – 3,9 GHz	—	—	504 MB	2025

Autres points de difficulté

Temps de compilation

Dépendances incontrôlable

Chaque programmeur utilise un sous ensemble du langage

Les programmes sont difficile à comprendre (difficultés de lecture, documentation pauvre, etc..)

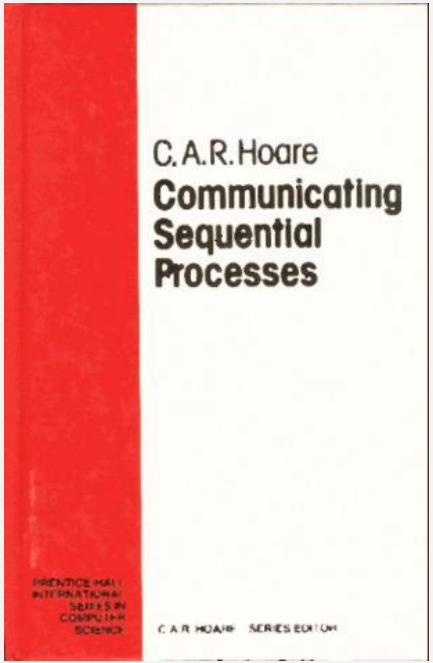
Duplication des efforts

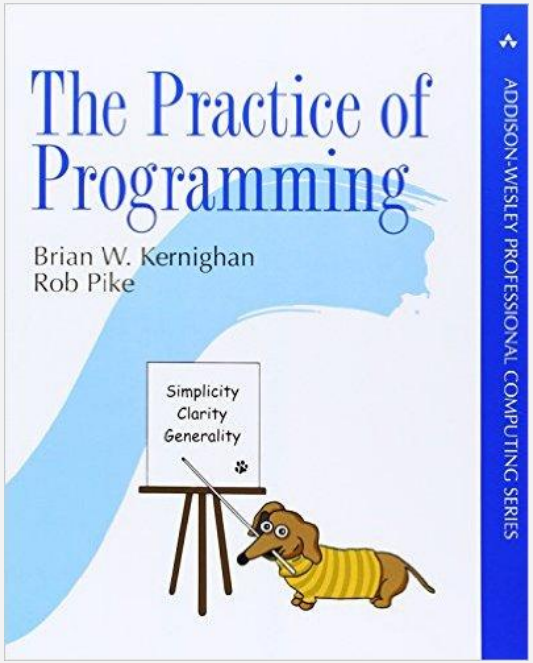
Des version qui cassent & Coût des mises à jours

Difficulté pour écrire des outils automatiques

compilation avec d’autres langages

Publications : la génèse de Go

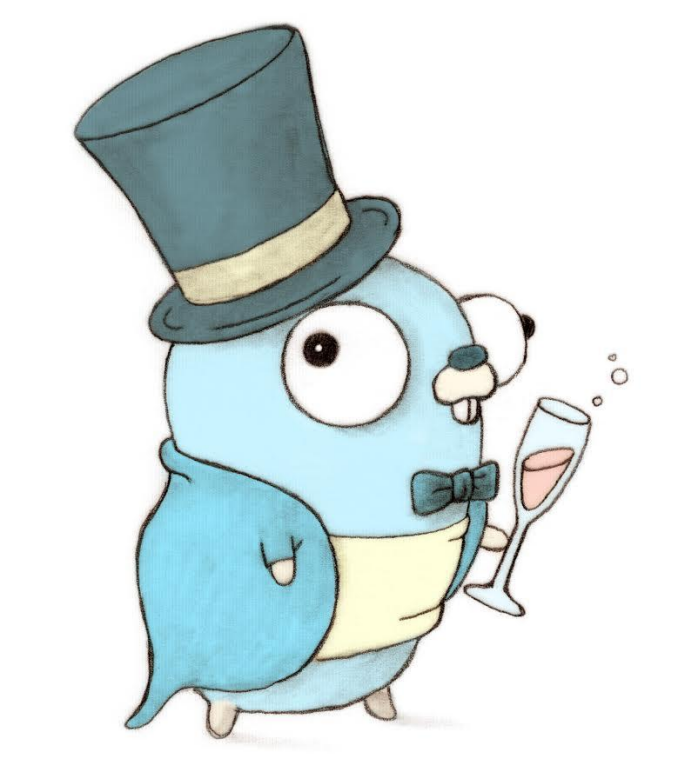




La solution : Go

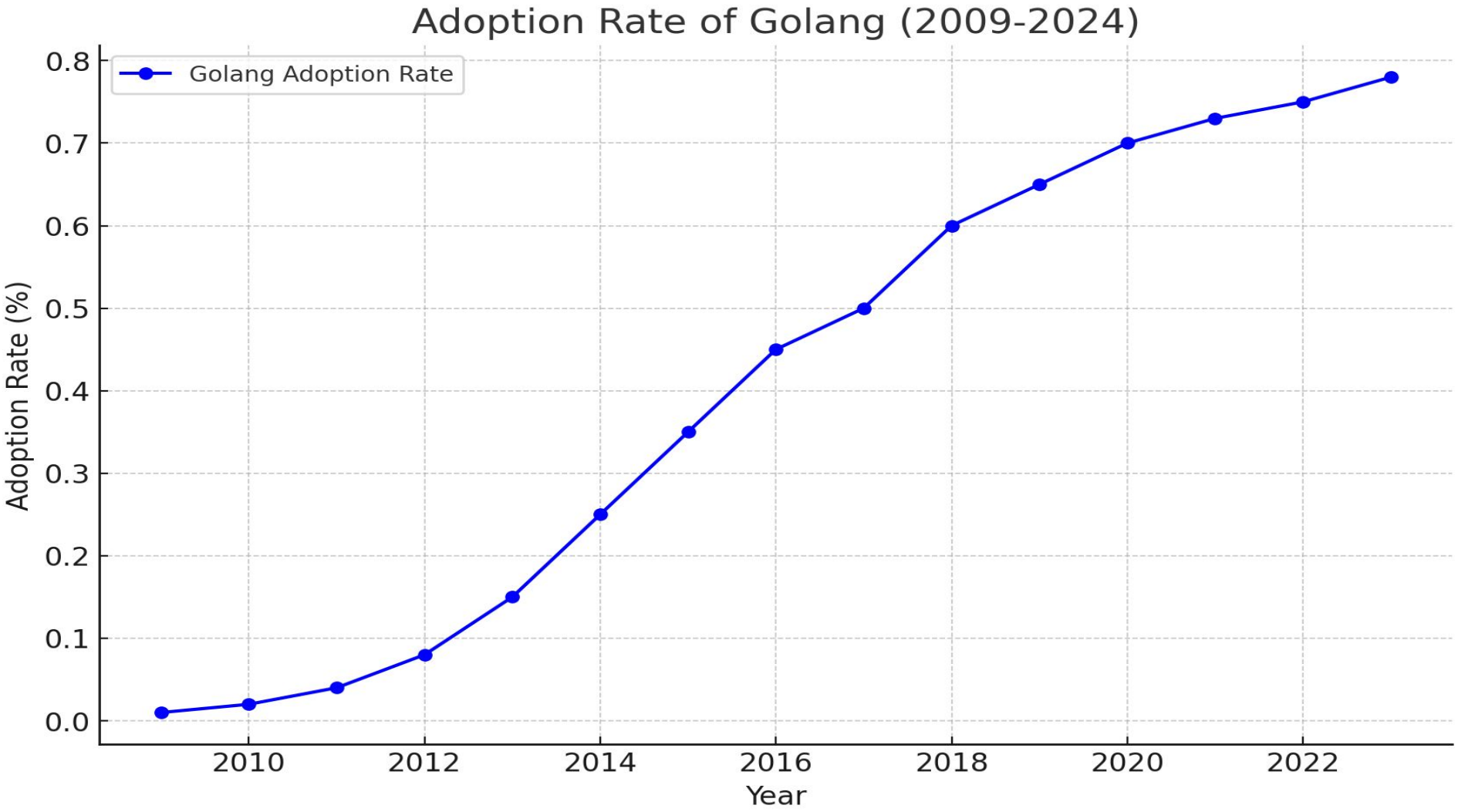
Aussi connue sous le nom de Golang, Go est un langage rapide d’exécution et flexible pour la réalisation de backend, d’API

Pour réaliser des applications robustes, Go est votre meilleur choix.



Go comme Google

Go team

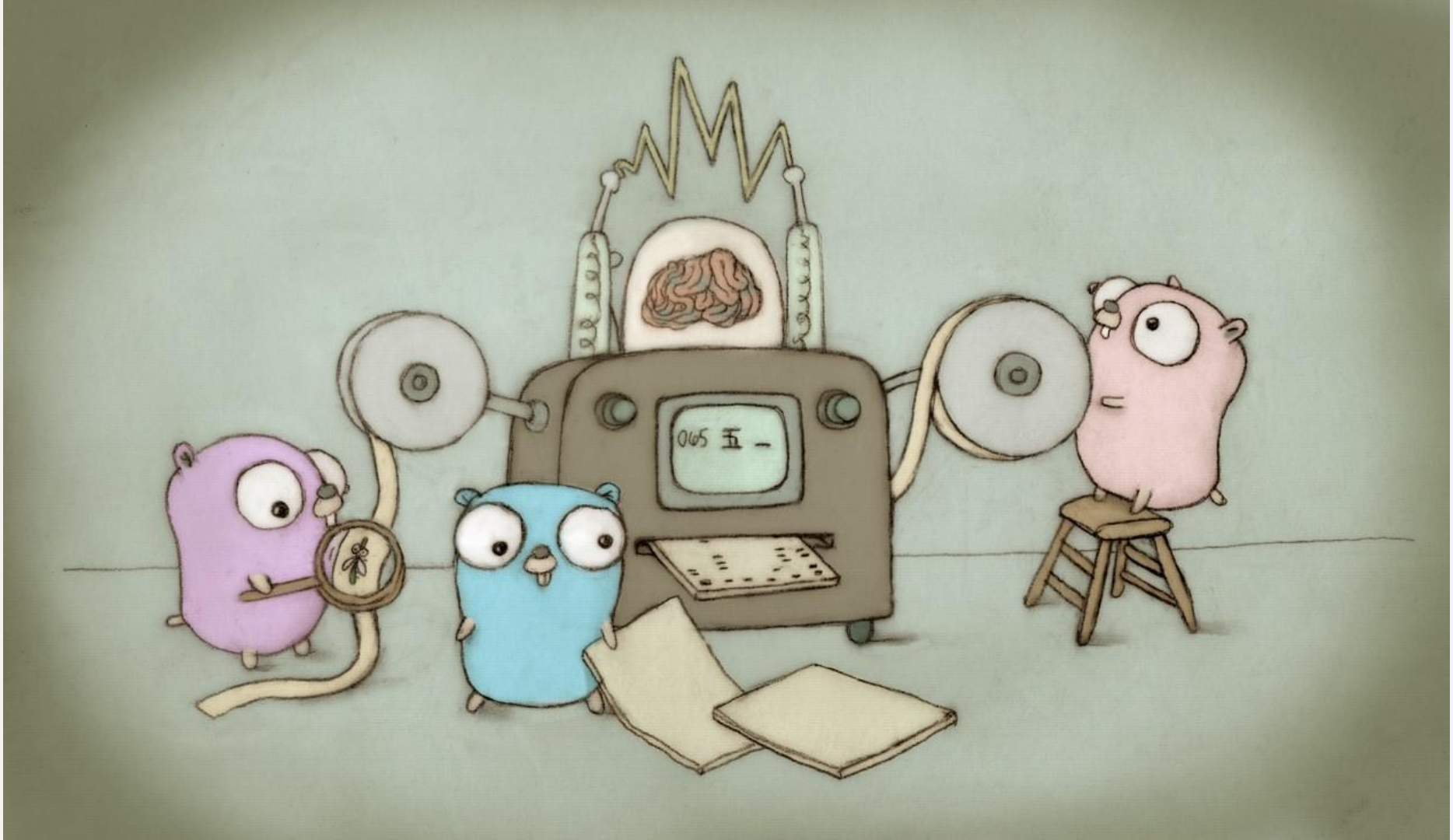

Adoption Rate of Golang (2009-2024)
Golang Adoption Rate
Adoption Rate (%)
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
2010
2012
2014
2016
2018
2020
2022
Year

Adoption de Go

Go Meetups

This is a list of all the groups in meetup.com that mention #golang as one of its topics. The list is ordered by number of members and you can apply filters to any column. The continent for each country is obtained using Freebase.

Name	City	Country	Members
filter	filter	filter	81096
Startup Edmonton	Edmonton	CA	3792
Seattle Google Developer Group (GDG)	Seattle	US	3589
GoSF	San Francisco	US	3403
Docker New York City	New York	US	2799
Pittsburgh Code &	Pittsburgh	US	2465


065 五 _

Gopher

Performance en 6 mois

The Computer Language Benchmarks Game

Go programs versus C++ g++

all the Go program measurements

by benchmark task performance

reverse-complement

source	secs	KB	gz	cpu	cpu load
Go	**0.53**	88,368	1278	0.94	28% 47% 77% 31%
C++ g++	0.59	219,048	2275	0.89	83% 21% 53% 5%

mandelbrot

source	secs	KB	gz	cpu	cpu load
Go	6.44	32,044	894	25.58	99% 100% 100% 100%
C++ g++	5.82	35,860	726	22.38	100% 95% 96% 95%

fasta-redux

source	secs	KB	gz	cpu	cpu load
Go	1.91	1,860	1236	1.90	1% 1% 100% 1%
C++ g++	1.66	1,620	1474	1.65	0% 100% 1% 2%

The Computer Language Benchmarks Game

Go programs versus C++ g++

all other Go programs & measurements

by benchmark task performance

reverse-complement

source	secs	KB	gz	cpu	cpu load
Go	**0.49**	88,320	1278	0.84	30% 28% 98% 34%
C++ g++	0.59	217,564	2275	0.84	26% 78% 12% 34%

mandelbrot

source	secs	KB	gz	cpu	cpu load
Go	**5.68**	30,756	894	22.56	100% 100% 99% 99%
C++ g++	5.82	33,952	726	22.40	96% 95% 95% 100%

fasta-redux

source	secs	KB	gz	cpu	cpu load
Go	1.82	1,384	1237	1.82	1% 100% 1% 0%
C++ g++	1.66	1,628	1474	1.65	100% 0% 0% 4%

Performance Go vs C

The Computer Language Benchmarks Game

Go programs versus C gcc

all other Go programs & measurements

by benchmark task performance

mandelbrot

source	secs	KB	gz	cpu	cpu load
Go	**5.68**	30,756	894	22.56	100% 100% 99% 99%
C gcc	5.92	32,572	694	22.78	95% 95% 95% 100%

fasta-redux

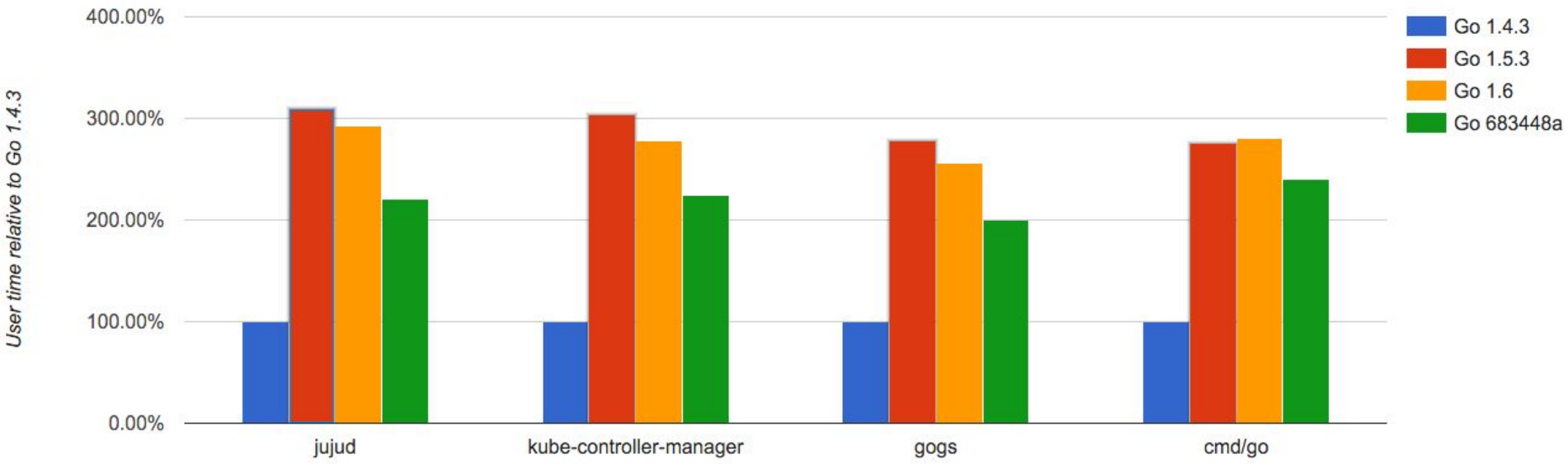
source	secs	KB	gz	cpu	cpu load
Go	1.82	1,384	1237	1.82	1% 100% 1% 0%
C gcc	1.55	780	1321	1.55	0% 100% 2% 1%

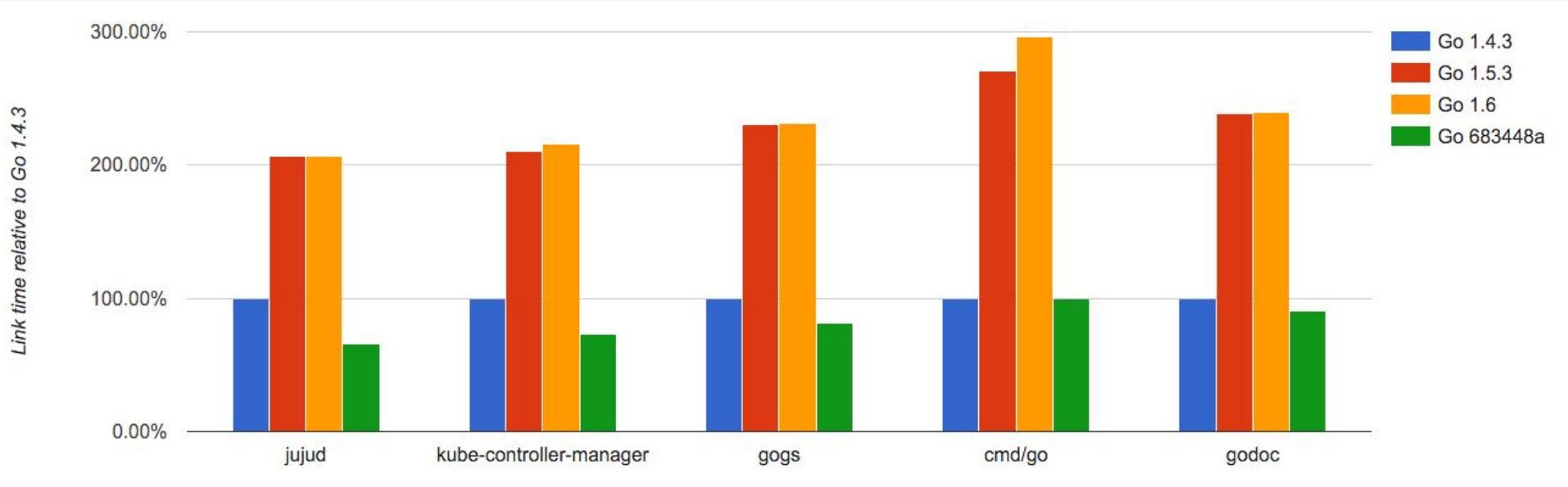
reverse-complement

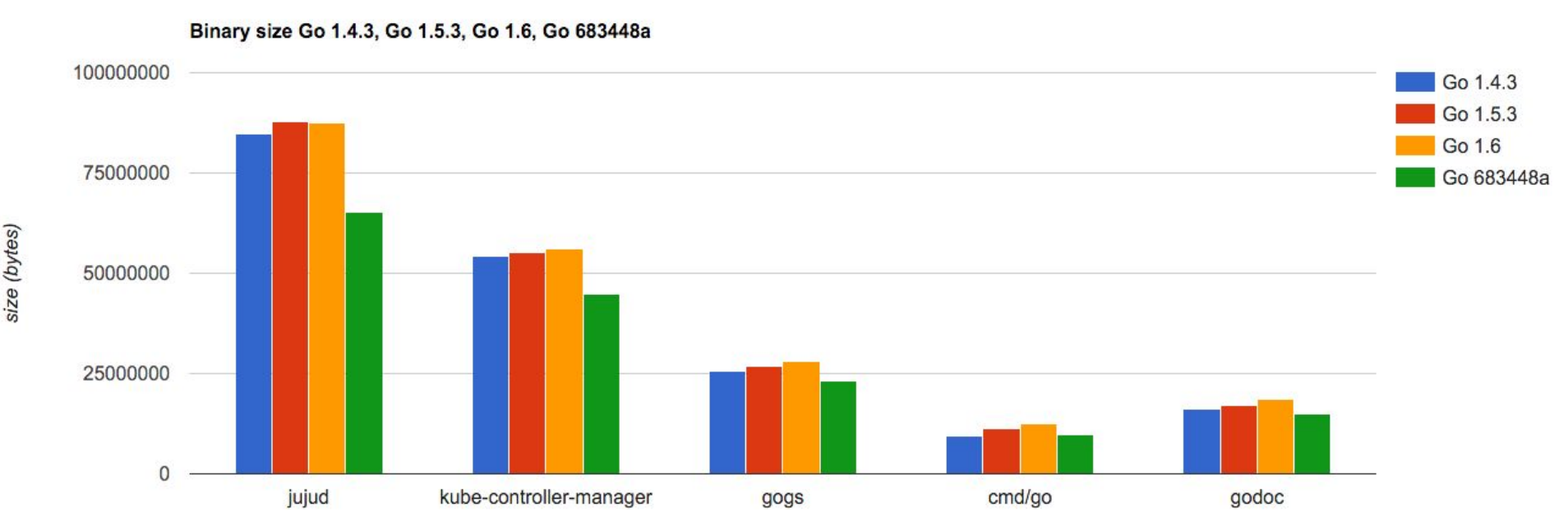
source	secs	KB	gz	cpu	cpu load
Go	0.49	88,320	1278	0.84	30% 28% 98% 34%
C gcc	0.42	145,900	812	0.57	0% 26% 20% 100%

fasta

source	secs	KB	gz	cpu	cpu load
Go	1.97	3,456	1344	5.76	76% 71% 74% 73%


User time relative to Go 1.4.3
400.00%
300.00%
200.00%
100.00%
0.00%
jujud
kube-controller-manager
gogs
cmd/go
Go 1.4.3
Go 1.5.3
Go 1.6
Go 683448a

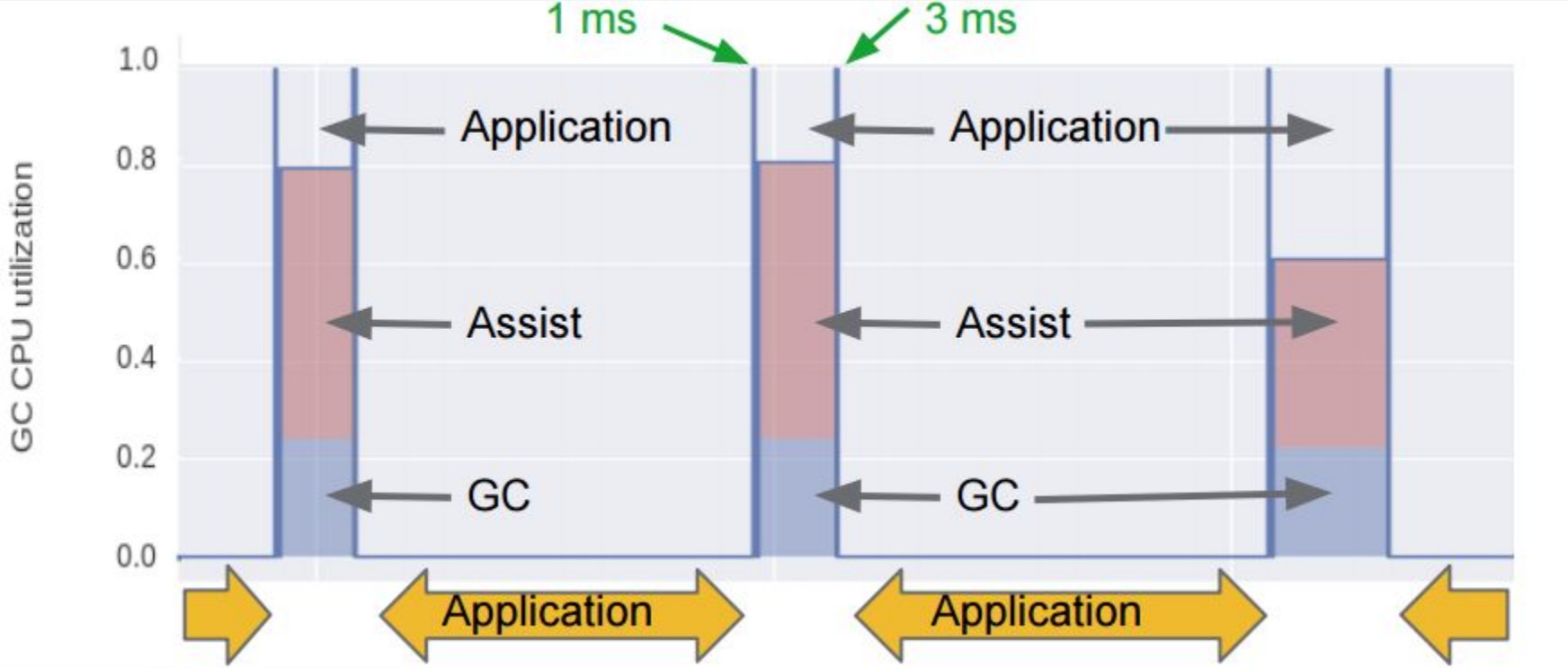

Link time relative to Go 1.4.3
300.00%
200.00%
100.00%
0.00%
jujud
kube-controller-manager
gogs
cmd/go
godoc
Go 1.4.3
Go 1.5.3
Go 1.6
Go 683448a

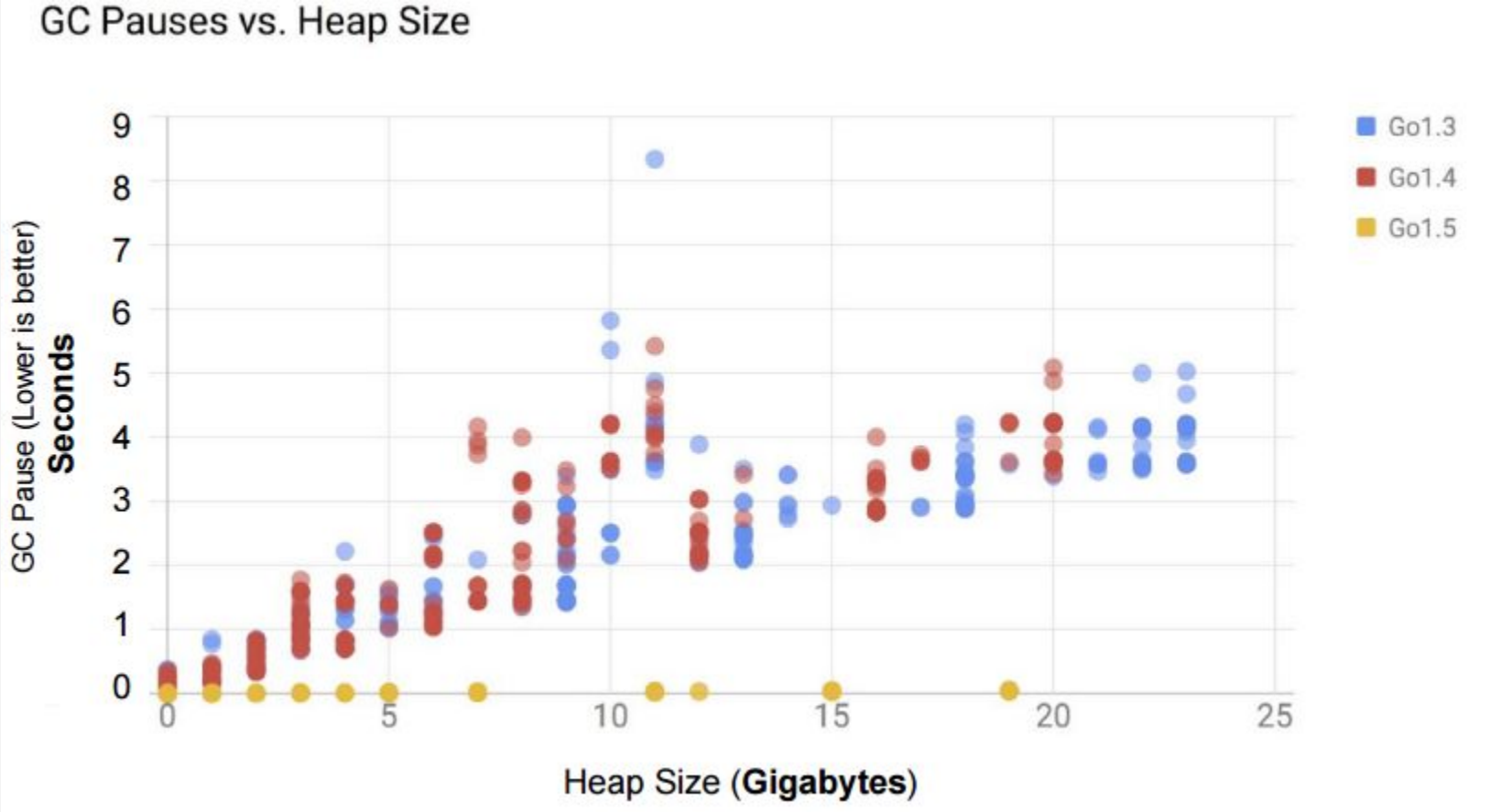

Binary size Go 1.4.3, Go 1.5.3, Go 1.6, Go 683448a
size (bytes)
100000000
75000000
50000000
25000000
0
jujud
kube-controller-manager
gogs
cmd/go
godoc
Go 1.4.3
Go 1.5.3
Go 1.6
Go 683448a

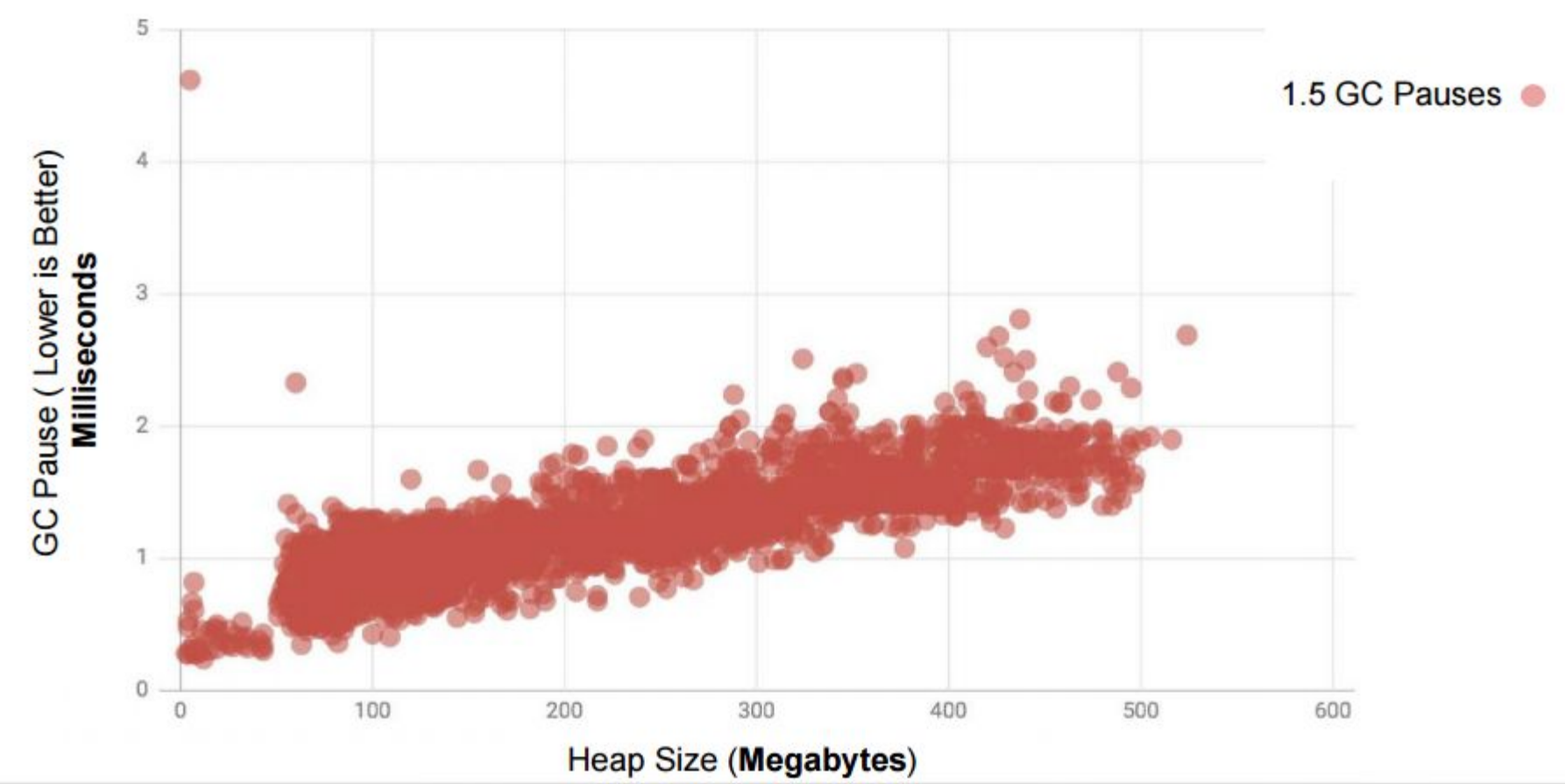
Garbage collector

Off			GC disabled Pointer writes are just memory writes: *slot = ptr
Stack scan	WB on		Collect pointers from globals and goroutine stacks Stacks scanned at preemption points
Mark	WB on		Mark objects and follow pointers until pointer queue is empty Write barrier tracks pointer changes by mutator
Mark termination	WB on	STW	Rescan globals/changed stacks, finish marking, shrink stacks, … Literature contains non-STW algorithms: keeping it simple for now
Sweep			Reclaim unmarked objects as needed Adjust GC pacing for next cycle
Off			Rinse and repeat

Garbage collector




GC Pauses vs. Heap Size
GC Pause (Lower is better)
Seconds
9
8
7
6
5
4
3
2
1
0
0
5
10
15
20
25
Heap Size (Gigabytes)
Go1.3
Go1.4
Go1.5



nouveau ramasse miettes

Quel avenir pour Go ?

Install party

Éléments à installer

Go	1.26
Go tools	vet, fmt, test...
IDE	VsCode
Plugins IDE	go-fmt, go-debug, tester-go